

Programmation dynamique

Sur les arbres

1. Introduction

1.1 Introduction

L'idée dans ce chapitre, c'est d'apprendre à obtenir des informations sur un arbre, en le parcourant de manière intelligente.

Definition 1.1.1 (Programmation dynamique (DP))

La programmation dynamique est une méthode de recherche qui consiste à résoudre un problème, en le décomposant en sous-problèmes, puis en calculant le résultat attendu à partir de ces sous-problèmes.

Dans cette méthode, on cherche à **mémoriser** le résultat de chaque sous-problème, afin que le résultat soit réutiliser ultérieurement.

Dans notre cas ici, nos sous-problèmes seront les enfants de chaque noeuds en général.

1.2 Vocabulaire

Par la suite, on considèrera que :

- les noeuds d'un arbre sont numérotés de 1 à n inclus.
- $\mathcal{T}$ comme étant un arbre enraciné au noeud 1.
- $\forall u \in \mathbb{N}$, nous avons $\mathcal{T}(u)$, l'ensemble des enfants du noeud u .

Pour répondre aux problèmes sur chaque noeud, on cherchera à stocker les résultats dans une liste dp de taille $n + 1$, où $dp[u]$ est le résultat attendu au noeud u .

Note

La taille de dp a été fixée à $n + 1$ car les noeuds sont numérotés de 1 à n inclus.

1.3 Technique

Dans un arbre $\mathcal{T}$, il est très facile de se concevoir comment faire de la programmation dynamique, car de manière générale, le résultat obtenu à partir des descendants d'un noeud u permettent de déduire le résultat attendu dans u .

Pour résoudre les problèmes, nous aurons besoin de passer par un *DFS* récursif pour traiter d'abord les descendants :

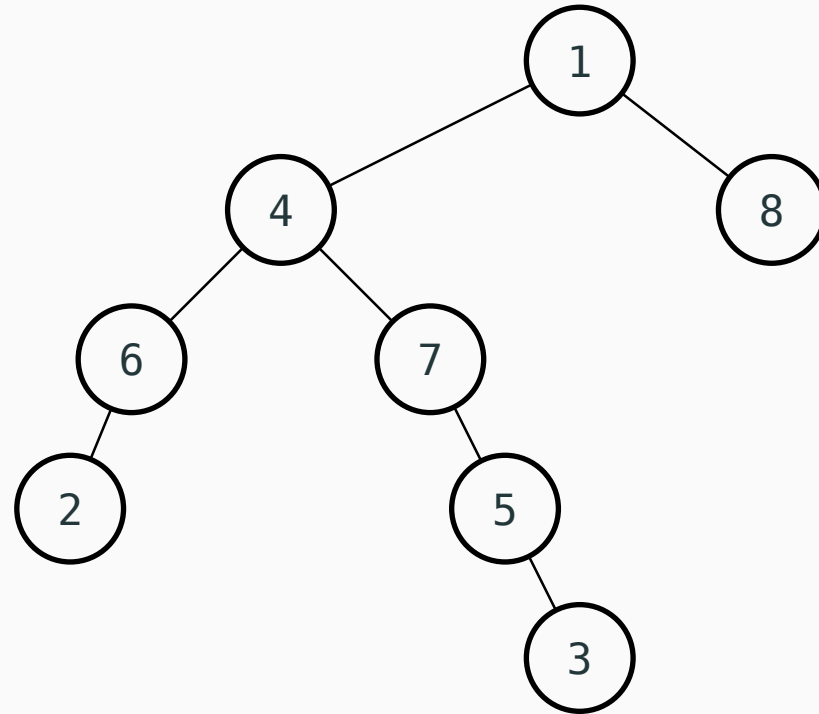
```
def dfs(u, tree):  
    for v in tree[u]:  
        dfs(v, tree)  
    # Traitement ici
```

2. Exemples classiques

2.1 Trouver les hauteurs de chaque noeud

Il est possible de trouver la hauteur de chaque noeud en $\mathcal{O}(n)$.

Voici l'arbre qu'on va utiliser comme exemple :



2.1 Trouver les hauteurs de chaque noeud

L'idée est d'étudier localement le problème.

Si on prend un noeud aléatoire, comment j'obtiens la hauteur du sous-arbre engendré par ce noeud, à partir de ceux engendrés par ses enfants ?

Rappel

La hauteur d'un noeud v est le nombre d'arêtes dans le chemin entre v , et la feuille la plus basse dans le sous-arbre obtenu à partir de v .

Etant donné qu'il s'agit de maximiser cette longueur, il est assez évident qu'on retrouve cette relation :

$$dp(u) = 1 + \max_{v \in T(u)} dp(v)$$

2.1 Trouver les hauteurs de chaque noeud

On suppose que `dp` est initialisé avec que des zéros, car une feuille a une hauteur nulle.

Nous allons faire usage de cette relation de transition, pour calculer les hauteurs à chaque noeud dans un *DFS* :

```
def dfs(u, tree, dp):  
    for v in tree[u]:  
        dfs(v, tree, dp)  
    dp[u] = max(dp[u], 1 + dp[v])
```

Note

En plus de cela, nous obtenons la hauteur de l'arbre qui est `dp[1]`.

2.2 Nombre de noeuds dans chaque sous-arbre

L'idée est similaire à l'exemple précédent, sauf qu'ici la relation de transition change :

$$dp(u) = 1 + \sum_{v \in T(u)} dp(v)$$

On considère que dp est initialisé avec que des uns.

Dans ce cas, on retrouve l'algorithme suivant :

```
def dfs(u, tree, dp):  
    for v in tree[u]:  
        dfs(v, tree, dp)  
    dp[u] += dp[v]
```

3. Plus bas ancêtre commun

3.1 Plus bas ancêtre commun

Soient u et v , deux noeuds de $\mathcal{T}$.

Definition 3.1.1 (Plus bas ancêtre commun)

L'**ancêtre commun le plus bas** (*Lowest Common Ancestor*) entre u et v , est le noeud w le plus profond qui contient u et v dans son sous-arbre.

Hormis pour faire de très beaux arbres généalogiques, cette notion a un usage très fréquent.

Par exemple pour :

- Trouver la distance entre n'importe quelle paire de noeuds dans un arbre.
- Reconstruire un cycle dans un graphe.

3.2 Trouver le LCA

Soient v et w , deux noeuds dans $\mathcal{T}$.

Dans un premier temps, nous allons générer une liste P , où pour tout noeud u , $P[u]$ est la **profondeur** du noeud u .

Ensuite, nous allons chercher à générer une liste A , où pour tout noeud u , $A[u]$ est l'ancêtre direct de u .

L'idée est de :

1. Equilibrer la profondeur de v et w , en remontant dans les ancêtres avec A .
2. Remonter dans les ancêtres en même temps, jusqu'à tomber sur un même noeud s .
3. s est donc le LCA de v et w

3.2 Trouver le LCA

```
def lca(u, v, ancestors, P):  
    while P[u] > P[v]: # u est plus profond que v  
        u = ancestors[u]  
  
    while P[v] > P[u]: # v est plus profond que u  
        v = ancestors[v]  
  
    while u != v: # On remonte en même temps  
        u = ancestors[u]  
        v = ancestors[v]  
  
    return u # lca
```

4. Crédits

Auteurs et images : Bilel MAHDJOUR

Pour le **Bureau des technologies** de **Polytech Nice Sophia**

5. Sources et ressources

- [CSES Book](#)